INF1636 — Programação Orientada a Objetos

Prof. Ivan Mathias Filho — PUC-Rio

Trabalho — Jogo Clue

Relatório da 1ª Iteração: Componente Model

Lucas Raposo 2320869, Hugo Freires 2321223, Vinícius Cortez 2210458

Maio de 2026

1. Introdução

Este relatório descreve a 1ª iteração da implementação do componente Model do jogo Clue, conforme as instruções do trabalho de INF1636 emitidas em 24/04/2026. O objetivo da iteração é entregar um pacote Java que ofereça, a outras camadas (View e Controller, ainda não desenvolvidas), os serviços mínimos para preparar uma partida e executar uma jogada: lançar dados, mapear casas alcançáveis e deslocar o pião do jogador da vez.

O documento expõe a estrutura do pacote, as decisões de design tomadas, a justificativa para cada uma delas, a relação com os princípios de orientação a objetos cobrados no enunciado (ocultação de informação e aberto/fechado), e a estratégia de testes unitários adotada com JUnit 4.

2. Visão Geral da Arquitetura

A arquitetura final do sistema é MVC. Nesta iteração apenas o Model existe; View e Controller serão construídos depois. Para que o trabalho futuro de Controller e View seja simples, o Model expõe uma API mínima e estável através de uma única classe pública: Jogo. Todas as demais classes do pacote model são package-private, isto é, o compilador Java impede que pacotes externos as referenciem.

A estrutura de pastas adotada é a seguinte:

```
inf1636-clue/
├── src/
│   ├── model/       (pacote Model — só Jogo é pública)
│   │   ├── Jogo.java
│   │   ├── Tabuleiro.java
│   │   ├── Casa.java
│   │   ├── Carta.java
│   │   ├── Envelope.java
│   │   ├── Baralho.java
│   │   ├── Jogador.java
│   │   ├── Pino.java
│   │   └── Dados.java
│   └── app/
│       └── Main.java        (demo — usa só model.Jogo)
├── test/
│   └── model/               (JUnit 4, dentro do pacote)
│       ├── DadosTest.java
│       ├── BaralhoTest.java
│       ├── EnvelopeTest.java
│       ├── TabuleiroTest.java
│       └── JogoTest.java
└── lib/
    ├── junit-4.13.2.jar
    └── hamcrest-core-1.3.jar
```

A classe Main vive num pacote separado (app) intencionalmente: serve de prova viva de que toda interação externa passa apenas pela classe pública Jogo. Tentar fazer Main importar Carta ou Tabuleiro provoca erro de compilação, exatamente o resultado pretendido.

3. Decisões de Design

3.1. Ocultação de informação (visibilidade package-private)

O enunciado é explícito: "todas as classes que representem elementos básicos do jogo, tais como o tabuleiro e as cartas, têm de ser declaradas como não públicas". Em Java, omitir o modificador public faz a classe ter visibilidade de pacote. Só código do mesmo pacote pode instanciá-la, herdá-la ou usar seus métodos.

A tabela abaixo lista a visibilidade adotada para cada classe e a razão:

Classe	**Visibilidade**	**Motivo**
Jogo	Público	Única porta de entrada do Model.
Tabuleiro	Package-private	Detalhe interno; representação pode mudar.
Casa	Package-private	Entidade auxiliar do Tabuleiro.
Carta	Package-private	Manipulação direta seria perigosa.
Envelope	Package-private	Guarda a solução do crime.
Baralho	Package-private	Encapsula a lógica de embaralhamento.
Jogador	Package-private	Exposto via métodos de Jogo.
Pino	Package-private	Peão é detalhe de Jogador.
Dados	Package-private	Mecanismo de aleatoriedade isolado.

Como consequência prática, métodos públicos de Jogo que precisam expor cartas devolvem List<String> em vez de List<Carta>, pois Carta não pode "vazar" do pacote. É uma decisão de limite: a representação de uma carta como objeto rico fica interna, o que sai do Model é apenas informação textual suficiente para a View desenhar.

3.2. Princípio Aberto/Fechado

Bertrand Meyer define um módulo "fechado" quando sua API é estável o bastante para outros módulos dependerem dela, e "aberto" quando ainda admite extensão sem alteração nessa API. Aplicado aqui:

A API pública (métodos de Jogo) descreve ações, como lancarDados(), mapearCasas(), deslocarPiao(), proximoJogador(). Isso a torna fechada para o Controller futuro.

Internamente, Casa pode ganhar atributos novos (tipo: corredor/sala/porta, cor, ocupante) sem alterar a assinatura de nenhum método de Jogo. Carta pode adicionar uma imagem associada. Tabuleiro pode trocar o grid por um grafo. O Model está aberto a essas evoluções.

A separação Jogador / Pino antecipa essa flexibilidade: Pino cuida só de posição; Jogador cuida só de identidade e mão. Acrescentar tabuleiros com salas (estado adicional do Pino) ou regras de mão diferentes (estado de Jogador) não causa interferência cruzada.

3.3. Framework de Coleções e uso de HashMap

O enunciado pede HashMap "sempre que possível". Os pontos onde HashMap traz benefício real (lookup O(1)) são:

Tabuleiro.casasPorId: mapeia o id (inteiro) de uma casa para o objeto Casa. O Controller futuro recebe um clique do usuário em forma de id e precisa resolvê-lo para a casa correspondente; um HashMap evita varrer o grid.

Baralho.pilhas: mapeia o enum Carta.Tipo para a respectiva lista de cartas (PERSONAGEM, ARMA, LOCAL). Permite acesso direto a cada categoria pela chave de tipo.

Tabuleiro.mapearCasasAlcancaveis: internamente usa um HashMap<Integer, Integer> distancia para o BFS, registrando a profundidade em que cada casa foi descoberta. Essa estrutura também elimina o "já visitada?" em O(1).

Outros pontos do código usam ArrayList (Jogador.mao, Baralho.restantes durante a distribuição) porque o que importa ali é manter ordem ou iterar sequencialmente, de forma que não há lookup por chave. Forçar HashMap nesses lugares seria pior, não melhor.

3.4. Preparação para Singleton + Façade (3ª iteração)

O enunciado adia explicitamente a aplicação formal dos padrões Singleton e Façade para a 3ª iteração. Mesmo assim, o desenho atual já segue o espírito de Façade: existe uma única classe pública que reúne, num único ponto de acesso, todos os serviços do subsistema. A transformação futura em Singleton consistirá em três alterações pequenas e localizadas:

Tornar o construtor privado.

Adicionar um campo estático INSTANCIA e um método estático getInstance().

Mover a inicialização atualmente feita no construtor para um método público novoJogo(List<String>) que reconfigure a instância.

Nota-se que nenhum dos clientes externos (Controller, View, ou os próprios testes) precisa mudar significativamente. Isso valida que a estrutura atual é, em termos de Meyer, uma boa base "fechada".

4. Implementação das Regras Exigidas

O enunciado exige a implementação de quatro grupos de funcionalidades. As seções a seguir descrevem cada uma, indicando as classes e métodos envolvidos.

4.1. Itens 4–7 da Preparação do manual do Clue

Esses itens determinam a montagem inicial da partida. A implementação foi distribuída entre as classes Baralho, Jogador e Jogo, conforme detalhado abaixo.

Item 4 (envelope-solução) e Item 5 (embaralhar e distribuir as cartas restantes):

Toda a lógica de separar as cartas em três grupos, sortear uma de cada para o envelope e embaralhar as 18 restantes está na classe Baralho. O construtor de Jogo chama:

```
java
this.envelope = baralho.sortearSolucao();   // Item 4
baralho.distribuir(jogadores);              // Item 5
```

Item 6 (folha de notas de investigador):

Cada jogador recebe sua própria folha de notas. Na classe Jogador, foi adicionado um campo private final List<String> folhaDeNotas, inicializado no construtor com todos os 21 itens oficiais (6 suspeitos, 6 armas, 9 cômodos). Métodos públicos riscarDaFolha(String item) e getFolhaDeNotas() permitem marcar e consultar os itens já eliminados. A criação dos jogadores já distribui as folhas.

Item 7 (ordem de jogo – Srta. Scarlet começa):

No construtor de Jogo, após criar a lista de jogadores, um laço localiza aquele cuja cor é "Vermelho" (a peça de Srta. Scarlet) e define indiceJogadorAtual com esse índice. Dessa forma, independentemente da ordem dos nomes passada pelo usuário, a primeira jogadora é sempre a Srta. Scarlet.

A distribuição em rodízio (índice módulo número-de-jogadores) faz com que, quando 18 cartas não dividem igualmente entre N jogadores, a sobra seja distribuída uma por jogador a partir do primeiro — exatamente o que o manual pede ("um a mais para alguns").

4.2. Lançamento virtual dos dados

Implementado pela classe Dados, encapsulando um java.util.Random. O método lancar() devolve um array de 2 inteiros, cada um sorteado em [1..6] via random.nextInt(6) + 1.

Decisão deliberada: foi adicionado um construtor que aceita uma seed. O construtor padrão é quem o Jogo realmente usa em produção (aleatoriedade verdadeira), mas o construtor com seed permite que testes futuros forcem sequências reprodutíveis sem mexer na produção. O custo dessa flexibilidade é uma única linha extra de código, e ela paga dividendos em testabilidade.

4.3. Mapeamento de casas alcançáveis (BFS)

É o coração algorítmico desta iteração. A partir da casa onde o pino do jogador está e do número total de passos (soma dos dois dados), Tabuleiro.mapearCasasAlcancaveis devolve a lista de ids das casas que podem ser destino do movimento.

A implementação é uma busca em largura (BFS) com limite de profundidade:

```
fila <- (origem, distancia=0)
enquanto fila não vazia:
   (atual, d) = fila.poll()
   se d == passos && d > 0:
      adicionar atual ao resultado
   se d == passos: continue
   para cada vizinha 4-conectada transitável e não visitada:
      registrar distância d+1
      enfileirar
```

Pontos importantes:

Vizinhança 4-conectada (cima, baixo, esquerda, direita) — o pino não move na diagonal.

Casas com transitavel == false (paredes) são ignoradas, demonstrando que o algoritmo respeita obstáculos.

A casa de origem nunca aparece no resultado — o jogador é obrigado a se mover.

O BFS garante que cada casa seja descoberta pela sua distância mínima; a parada por profundidade evita expansão desnecessária.

Observação: a implementação atual garante que o jogador só pode parar após andar exatamente o número de passos (distância mínima). Contudo, o movimento ainda é feito pelo caminho mais curto (BFS), sem permitir desvios para gastar passos extras. A regra oficial que permite percursos mais longos (sem repetir casas) será implementada na próxima iteração, assim como as passagens secretas e portas.

4.4. Deslocamento do pião

O método Jogo.deslocarPiao(int idCasa) é simples mas valida três coisas antes de mover:

- Se os dados foram lançados nesta jogada (caso contrário, não há "alcance" definido — lança IllegalStateException).
- Se a casa solicitada está no resultado de mapearCasas para o último lançamento (caso contrário, lança IllegalArgumentException).
- Após mover, o lançamento é "consumido" (zerado), exigindo um novo lançarDados antes da próxima jogada.

Essas verificações deixam o estado do jogo correto mesmo se o Controller chamar a API fora de ordem — proteger contra mau uso é parte da contratação fechada da Façade.

5. Testes Unitários (JUnit 4)

Foram escritos 30 testes distribuídos em 5 classes, um para cada classe não trivial do Model. Todos passam. As classes de teste vivem no mesmo pacote (model) que o código sob teste — exigência prática quando se respeita a visibilidade package-private, pois testes precisam acessar membros que não são públicos.

Classe	**O que cobre**
DadosTest (3)	Domínio de saída [1..6], comprimento 2, reprodutibilidade com seed.
BaralhoTest (5)	Tipos no envelope, contagem de cartas distribuídas (18), balanceamento, ausência de duplicatas e caso de 3 jogadores (6 cartas cada).
EnvelopeTest (5)	verificarSolucao com acusação correta e errada, e validação de tipo no construtor (3 casos de exceção esperada).
TabuleiroTest (6)	Limite de vizinhança 1-passo, exclusão da origem, paredes nunca listadas, posições iniciais distintas, casos extremos (0 passos, 2 passos).
JogoTest (11)	Integração: criação do jogo, validação de número de jogadores (3-6), fluxo completo das três etapas, controle de erro (deslocar antes de lançar / casa fora de alcance), rodízio de jogadores, e verificação da regra de início com Srta. Scarlet (testScarletSempreComeca).

Todos os testes seguem as diretrizes do Capítulo 18: nomes autoexplicativos, um comportamento por método, uso de assertEquals/assertTrue, e @Test(expected=...) para exceções.

6. Limitações Conhecidas e Próximos Passos

Funcionalidades implementadas e testadas:

- Itens 4-7 da Preparação (envelope-solução, embaralhar, distribuir).
- Lançamento de dois dados.
- Mapeamento por BFS de casas alcançáveis (com paredes).
- Deslocamento do pino com validação.
- Rodízio de turnos.

Limitações desta iteração (assumidas conscientemente):

- O tabuleiro é um grid 10x10 com algumas paredes: não o mapa real do Clue com salas, portas e passagens secretas. A modelagem permite essa evolução sem mudar a API.
- O mapeamento atual exige distância mínima exata (d == passos), mas ainda não permite percursos mais longos com desvios (o que seria possível no jogo real). Na próxima iteração, o algoritmo será evoluído para uma busca com backtracking que aceite caminhos não mínimos e trate as regras de passagens secretas e portas.
- Não há ainda os conceitos de palpite/sugestão e acusação: só a infraestrutura (Envelope.verificarSolucao já está pronta para acusação).
- Singleton + Façade ficarão para a 3ª iteração.

7. Como Compilar e Executar

A partir da pasta inf1636-clue/, na linha de comando:

```
# 1. Compilar o código de produção
mkdir -p bin
find src -name "*.java" | xargs javac -d bin -encoding UTF-8

# 2. Executar o demo
java -cp bin app.Main

# 3. Compilar os testes
mkdir -p bin-test
find test -name "*.java" | xargs javac -d bin-test \
   -cp "bin:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
   -encoding UTF-8

# 4. Rodar os testes
java -cp "bin:bin-test:lib/junit-4.13.2.jar:lib/hamcrest-core-1.3.jar" \
   org.junit.runner.JUnitCore model.JogoTest
```

No Eclipse: importar como projeto Java existente, marcar src/ e test/ como source folders, adicionar os JARs de lib/ ao build path e rodar JUnit pelo botão direito.

8. Conclusão

Esta 1ª iteração entrega um componente Model funcional, testado e preparado para crescer. As escolhas de visibilidade fecham o Model atrás de uma API pública estreita; a separação de responsabilidades (Tabuleiro vs. Casa, Jogador vs. Pino, Baralho vs. Envelope) e o uso de coleções apropriadas (HashMap onde há lookup, ArrayList onde há ordem) deixam o código pronto para receber novas regras sem reescrita.

Os 30 testes JUnit cobrem todas as funcionalidades exigidas no enunciado, incluindo as regras de movimento exato, a folha de notas e o início obrigatório pela peça vermelha, e servem de rede de segurança para as próximas iterações, quando serão implementados o Controller, a View, o suporte a desvios no movimento (backtracking) e as regras de palpite/acusação.